

Принципы SOLID

Содержание урока

- ★ Что такое SOLID
- ★ Single Responsibility
- ★ Open-Close
- ★ Liskov Substitution
- ★ Interface Segregation
- ★ Dependency Inversion

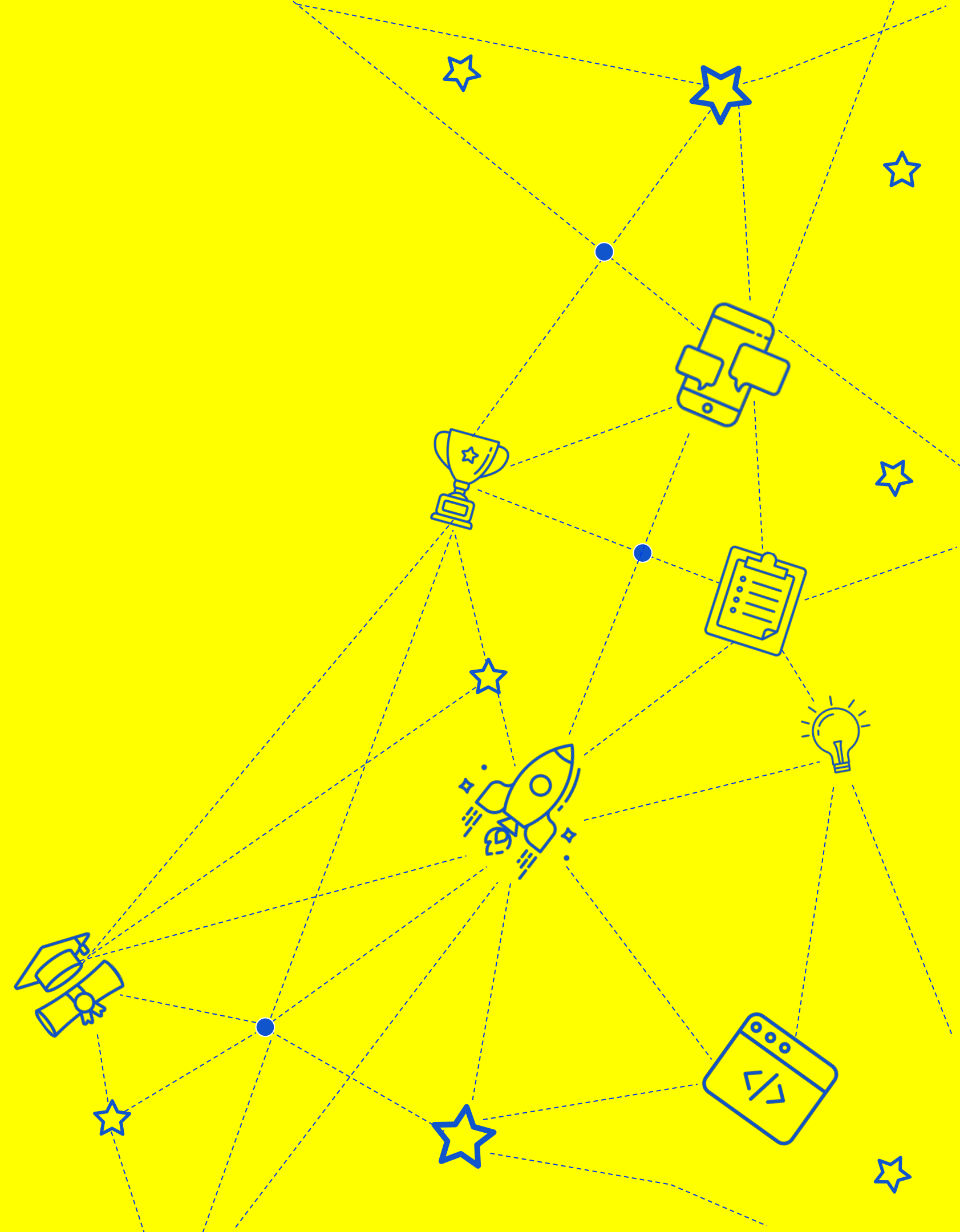


ИГОРЬ КРИВОНОС

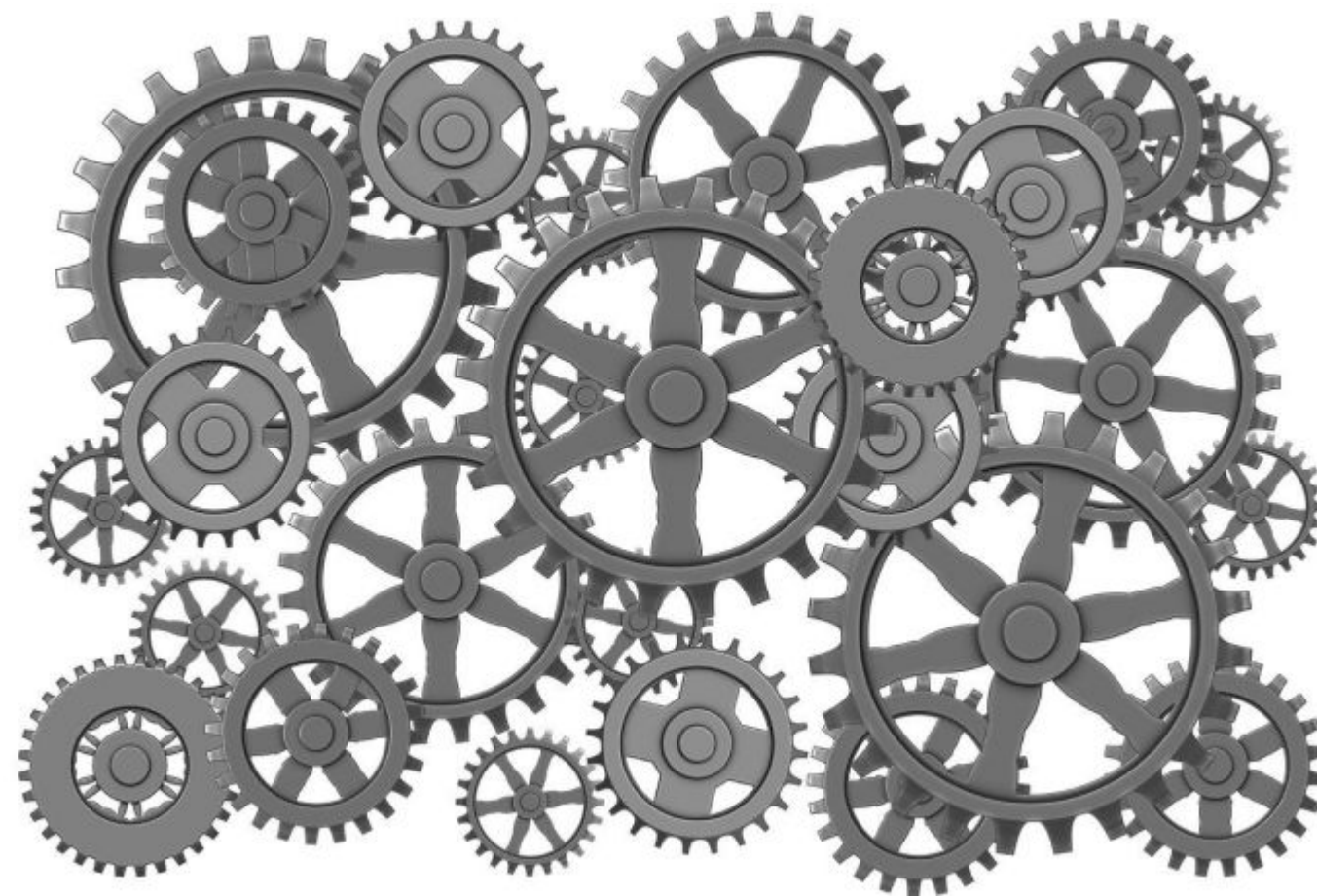
Senior Backend Software Engineer

- Консультирую стартапы по построению масштабируемой архитектуры приложения
- Занимаюсь менторингом более 7 лет
- Проектирую и реализовываю backend api с 2014 года

Что такое SOLID



Немного истории



Немного истории



Начало 2000-х

Роберт Мартин предложил свод правил, которые были призваны сделать разработку ПО проще

А **Майкл Фэзерс** немного видоизменил их и ввел мнемонический акроним SOLID для простоты запоминания

Что такое SOLID



1 **Single Responsibility** — принцип единственной ответственности

2 **Open-Close** — принцип открытости-закрытости

3 **Liskov Substitution** — принцип подстановки Барбары Лисков

4 **Interface Segregation** — принцип разделения интерфейсов

5 **Dependency Inversion** — принцип инверсии зависимостей



Какие преимущества дает SOLID

01

Простота поддержки

02

Простота расширения
существующего функционала

03

Простота разработки
нового функционала

04

Стабильное развитие системы
в течение долгого времени

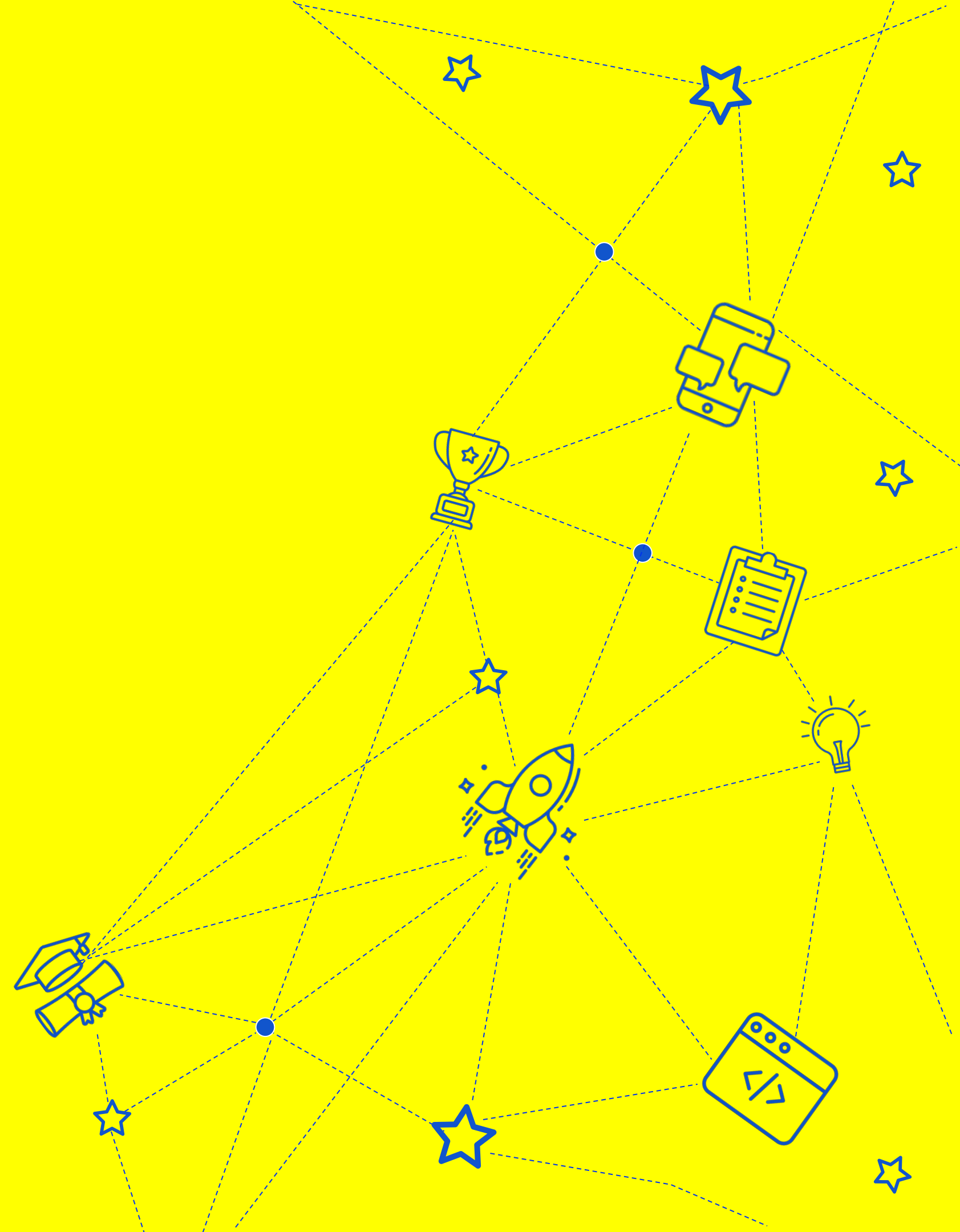
Недостатки SOLID

01 Больше кода

02 Нужно «думать»



Single Responsibility



Single Responsibility



Принцип единственной ответственности
(Single Responsibility Principle) —
для каждого класса должно быть
определено единственное назначение

Все ресурсы, необходимые для
его осуществления, должны быть
инкапсулированы в этот класс
и подчинены только этой задаче



Сложно? Можно проще

Принцип единственной ответственности — у компонента должна быть одна строго определенная роль. Функционал, который к этой роли не относится, должен быть в другом классе. Иными словами, у вас должна быть только одна причина его менять



Погрузимся в детали

Хороший пример

- ★ Класс ответственный за валидацию входных сообщений
- ★ Класс авторизации пользователя
- ★ Класс процессинга создания заказа в интернет магазине

Плохой пример

- ★ Класс подсчета суммы заказа и конвертации в другую валюту
- ★ Класс поиска ошибок в тексте и форматирования его по стандартам

А теперь к коду

```
from pathlib import Path
from zipfile import ZipFile
```

Роль класса – управлять одним конкретным файлом

```
class FileManager:
    def __init__(self, filename):
        self.path = Path(filename)
```

```
    def read(self, encoding="utf-8"):
        return self.path.read_text(encoding)
```

```
    def write(self, data, encoding="utf-8"):
        self.path.write_text(data, encoding)
```

Чтение и запись в различных кодировках — роль вложенного класса

```
    def compress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="w") as archive:
            archive.write(self.path)
```

```
    def decompress(self):
        with ZipFile(self.path.with_suffix(".zip"), mode="r") as archive:
            archive.extractall()
```

Неочевидный постфикс

Архивация и деархивация нарушает принцип единственной ответственности

А как правильно?

Роль класса — управлять одним конкретным файлом



```
class FileManager:
    def __init__(self, filename):
        self.path = Path(filename)

    def read(self, encoding="utf-8"):
        return self.path.read_text(encoding)

    def write(self, data, encoding="utf-8"):
        self.path.write_text(data, encoding)
```

Роль класса — управлять одним конкретным архивированным файлом



```
class ZipFileManager:
    def __init__(self, filename):
        self.path = Path(filename)

    def compress(self):
        with ZipFile(self.path, mode="w") as archive:
            archive.write(self.path)

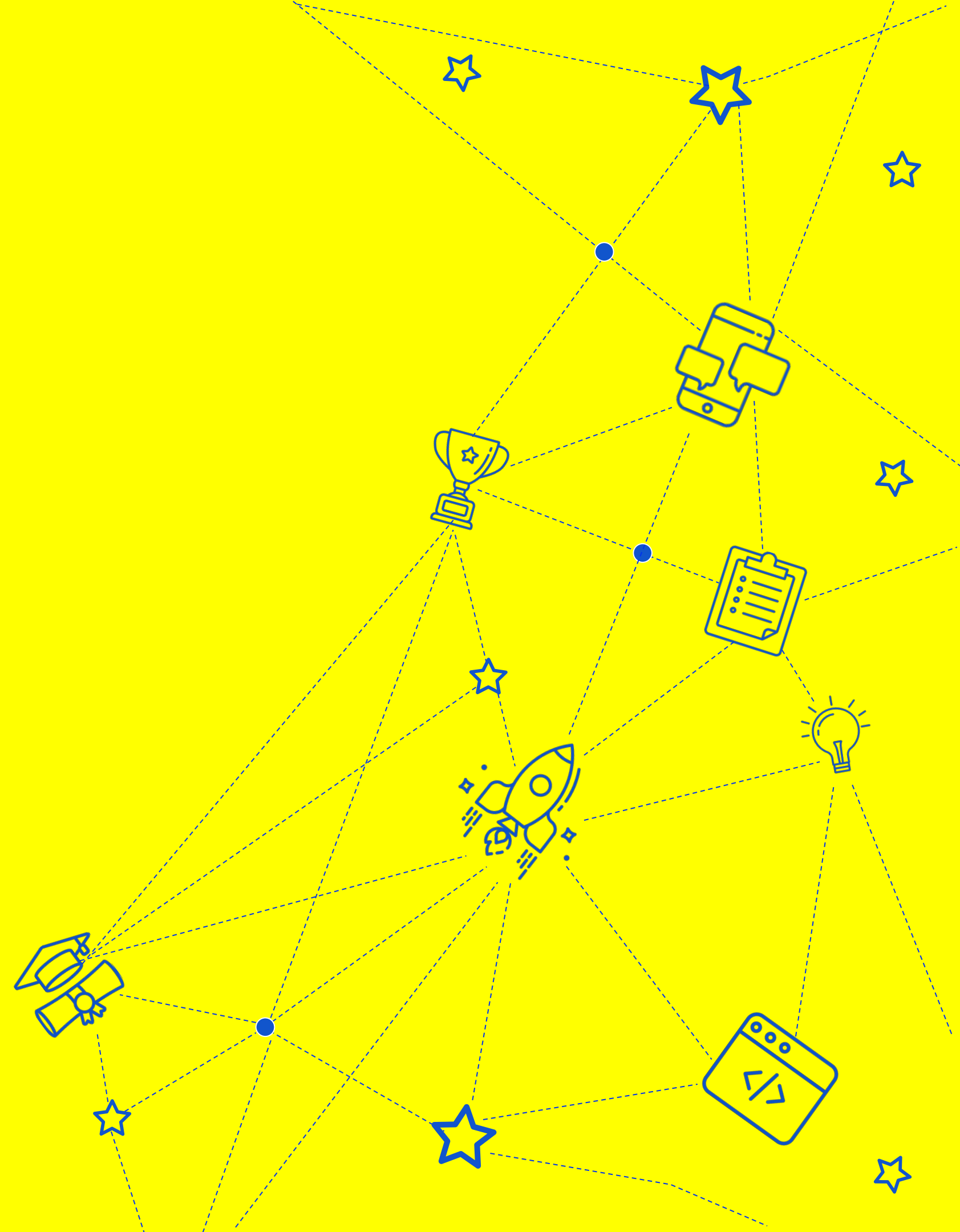
    def decompress(self):
        with ZipFile(self.path, mode="r") as archive:
            archive.extractall()
```


Что вам это даст



1. Функционал класса можно расширять, не боясь сломать старый код
2. Чтение, запись, архивацию и разархивацию можно переиспользовать в других классах
3. Простота чтения
4. Тестируемость

Open-close



Open-close



Принцип открытости-закрытости
(open-close principle) — «программные
сущности должны быть открыты для
расширения, но закрыты для модификации»

Бертран Мейер



Сложно? Можно проще

Принцип открытости-закрытости — для добавления нового функционала должно быть легко дополнить код чем-то новым и сложно заменить существующий



Погрузимся в детали

Хороший пример

- ★ Добавление нового метода через создание нового дочернего класса
- ★ Через наследование добавить дополненное поведение методу

Плохой пример

- ★ Менять код уже существующего класса, чтобы добавить новый функционал без изменения бизнес требований

А теперь к коду

```
class Shape:
    def __init__(self, shape_type, **kwargs):
        self.shape_type = shape_type
        if self.shape_type == "rectangle":
            self.width = kwargs["width"]
            self.height = kwargs["height"]
        elif self.shape_type == "circle":
            self.radius = kwargs["radius"]

    def calculate_area(self):
        if self.shape_type == "rectangle":
            return self.width * self.height
        elif self.shape_type == "circle":
            return pi * self.radius**2
```

Чтобы добавить новый тип
фигуры, нужно изменить
уже написанный код

А как правильно?

```
class Shape(ABC):  
    def __init__(self, shape_type):  
        self.shape_type = shape_type  
  
    @abstractmethod  
    def calculate_area(self):  
        pass
```

!

Для добавления нового типа фигуры нужно написать новый класс, а старые останутся неизменными

```
class Circle(Shape):  
    def __init__(self, radius):  
        super().__init__("circle")  
        self.radius = radius  
  
    def calculate_area(self):  
        return pi * self.radius**2  
  
class Rectangle(Shape):  
    def __init__(self, width, height):  
        super().__init__("rectangle")  
        self.width = width  
        self.height = height  
  
    def calculate_area(self):  
        return self.width * self.height
```

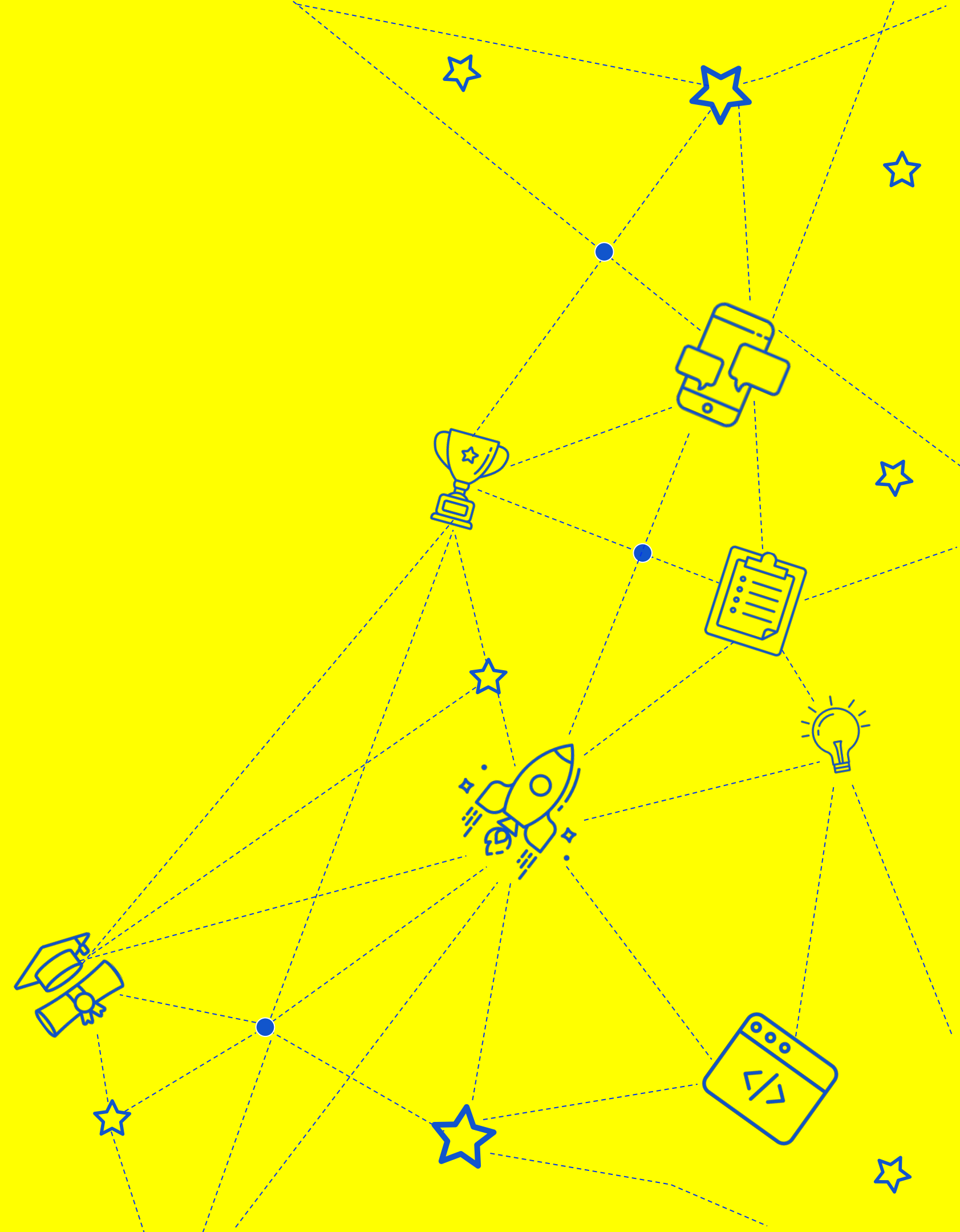
Что вам это даст

Старый функционал
гарантированно
не сломается



Нужно тестировать
только новый
функционал

Liskov Substitution Principle



Liskov Substitution Principle



Принцип подстановки Барбары Лисков
(Liskov Substitution Principle) — «функции,
которые используют базовый тип, должны
иметь возможность использовать подтипы
базового типа, не зная об этом»

Роберт С. Мартин



Сложно? Можно проще

Принцип подстановки Барбары Лисков — дочерние классы можно использовать там же, где и родительские, без затруднений



Погрузимся в детали

Хороший пример

- ★ Дочерний класс добавляет логирование
- ★ Дочерний класс добавляет новые методы, не меняя старые

Плохой пример

- ★ Родительский класс — массив
- ★ Дочерний класс — сортированный массив

А теперь к коду

```
class Array:
    def __init__(self):
        self._items = []

    def put(self, new_item):
        self._items.append(new_item)

    def get_by_index(self, index):
        self._items[index]

class SortedArray(Array):
    def __init__(self):
        super().__init__()

    def put(self, new_item):
        self._items.append(new_item)
        self._items.sort()
```

Нарушен ли принцип?

```
arr = Array()
arr.put(1)
arr.put(3)
arr.put(2)

if arr.get_by_index(1) == 3:
    print('ok')
```

```
arr = SortedArray()
arr.put(1)
arr.put(3)
arr.put(2)
```

```
if arr.get_by_index(1) == 3:
    print('ok')
```

Условие не будет
выполнено



А как правильно?

```
class Collection(ABC):  
    def __init__(self):  
        self._items = []  
  
    @abstractmethod  
    def put(self, new_item):  
        pass  
  
    def get_by_index(self, index):  
        self._items[index]
```

```
class SortedArray(Collection):  
    def __init__(self):  
        super().__init__()  
  
    def put(self, new_item):  
        self._items.append(new_item)  
        self._items.sort()  
  
class Array(Collection):  
    def __init__(self):  
        super().__init__()  
  
    def put(self, new_item):  
        self._items.append(new_item)
```

Что вам это даст

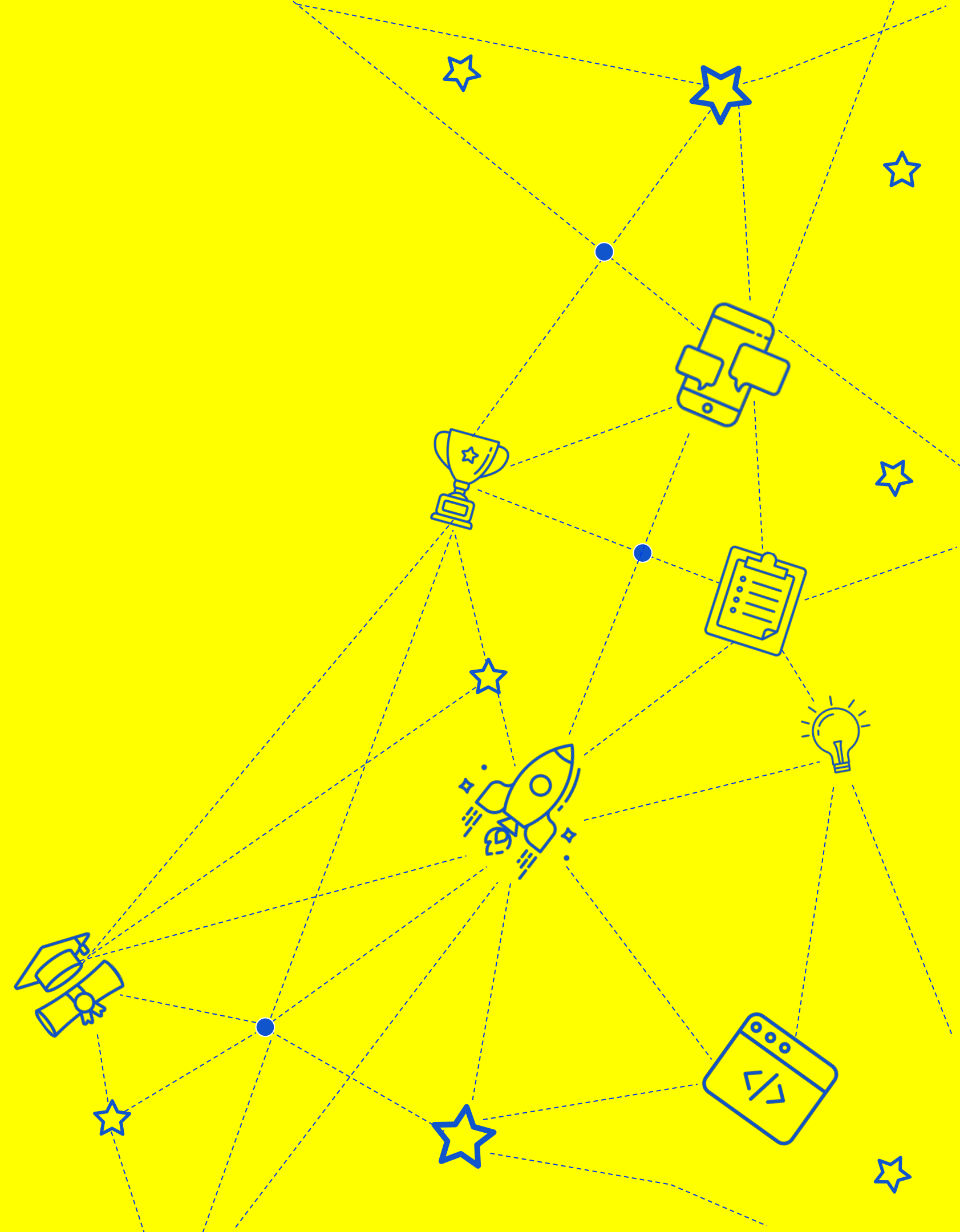
Расширяя классы через наследование, мы можем использовать их наследников в старом коде

Высокая переиспользуемость кода

Сложно загнать себя в ловушку



Interface Segregation



Interface Segregation



Принцип разделения интерфейса (Interface Segregation Principle) — «много интерфейсов, специально предназначенных для клиентов, лучше, чем один интерфейс общего назначения»



Сложно? Можно проще

Принцип разделения интерфейсов чем-то похож на Single Responsibility, но для интерфейсов. Лучше много отдельных интерфейсов, которые выполняют одну маленькую роль, чем один, выполняющий много ролей



Погрузимся в детали

Хороший пример

- ★ Интерфейс для чтения файлов
- ★ Интерфейс для записи файлов
- ★ Интерфейс для сериализации в JSON
- ★ Интерфейс для сериализации в XML

Плохой пример

- ★ Интерфейс для манипуляции файлом
- ★ Интерфейс сериализации в различные форматы

А теперь к коду

```
class Printer(ABC):
    @abstractmethod
    def print(self, document):
        pass

    @abstractmethod
    def scan(self, document):
        pass

class OldPrinter(Printer):
    def print(self, document):
        print(f"Printing {document} in black and white...")

    def scan(self, document):
        raise NotImplementedError("Scan functionality not supported")

class ModernPrinter(Printer):
    def print(self, document):
        print(f"Printing {document} in color...")

    def scan(self, document):
        print(f"Scanning {document}...")
```

Мы вынуждены реализовывать метод, который нам не нужен



А как правильно?

```
class Printer(ABC):  
    @abstractmethod  
    def print(self, document):  
        pass
```

```
class Scanner(ABC):  
    @abstractmethod  
    def scan(self, document):  
        pass
```

```
class OldPrinter(Printer):  
    def print(self, document):  
        print(f"Printing {document} in black and white...")
```

Реализовано
только то,
что нужно

```
class ModernPrinter(Printer, Scanner):  
    def print(self, document):  
        print(f"Printing {document} in color...")  
  
    def scan(self, document):  
        print(f"Scanning {document}...")
```

Реализовано
только то,
что нужно

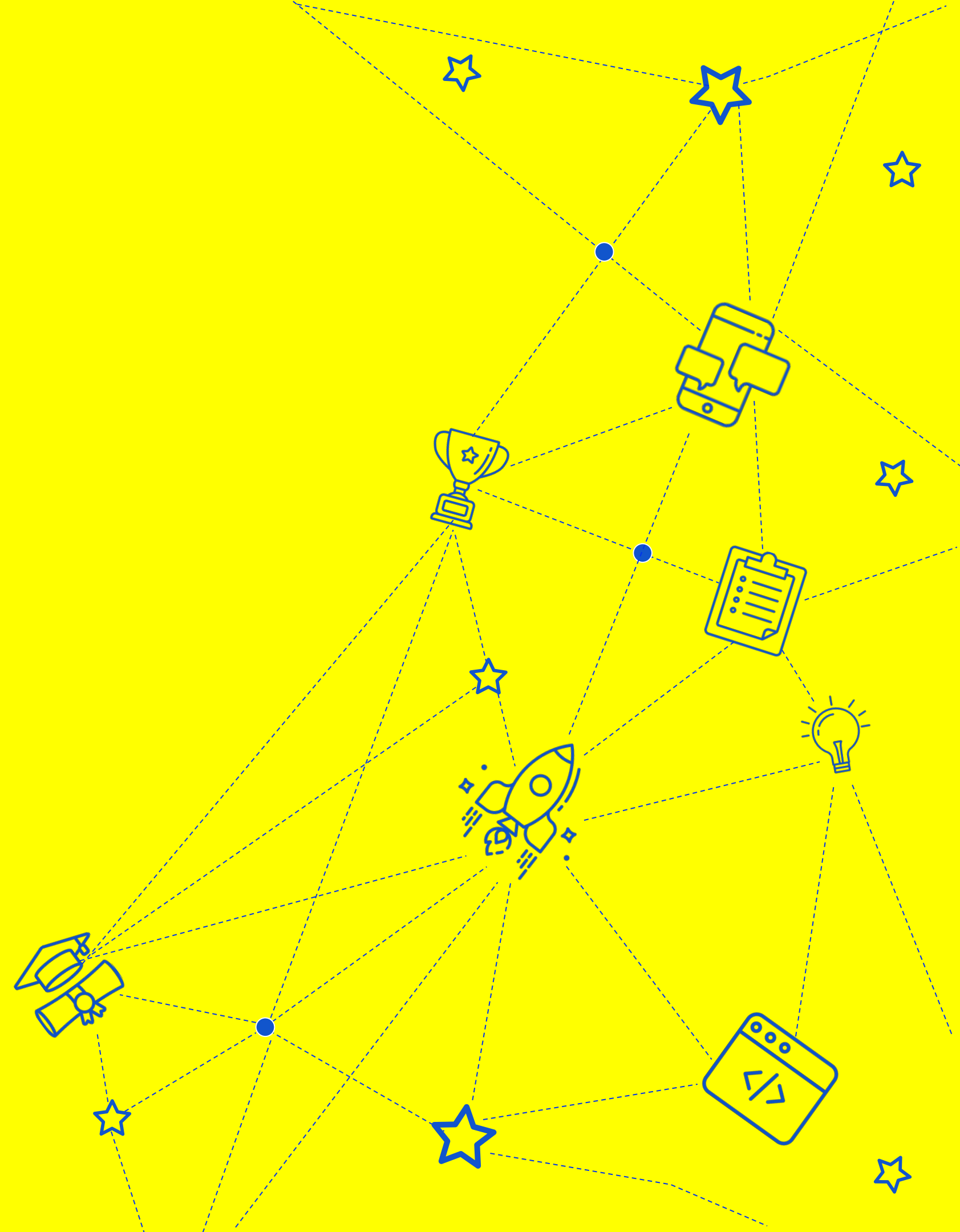
Что вам это даст

Не нужно реализовывать методы,
которые не нужны класу

Высокая переиспользуемость кода



Dependency Inversion



Dependency Inversion

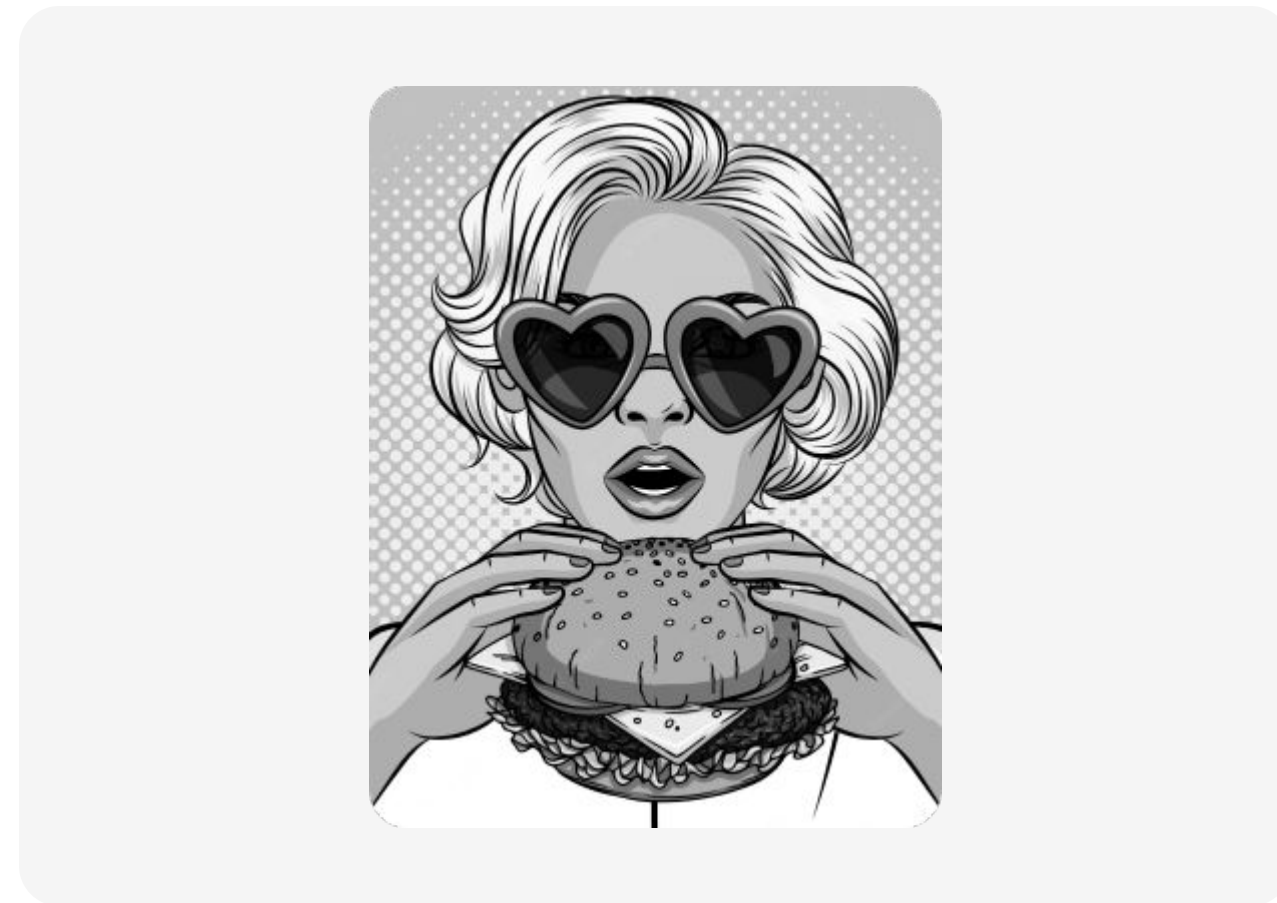


Принцип инверсии зависимостей
(Dependency Inversion Principle) — «зависимость
от абстракций, нет зависимости от конкретики»



Сложно? Можно проще

Принцип инверсии зависимостей — важно, чтобы ваши классы использовали интерфейсы (зависели от них), а не от одного единственного класса реализации



Погрузимся в детали

Хороший пример

- ★ Класс работы с файлами зависит от интерфейса файлового менеджера

Плохой пример

- ★ Класс работы с файлами зависит от класса файлового менеджера для работы с файловой системой

А теперь к коду

```
class FrontEnd:
    def __init__(self, back_end):
        self.back_end = back_end

    def display_data(self):
        data = self.back_end.get_data_from_database()
        print("Display data:", data)

class BackEnd:
    def get_data_from_database(self):
        return "Data from the database"
```

FrontEnd может брать данные только из базы данных, для замены источника — нужно менять код



А как правильно?

```
class FrontEnd:
    def __init__(self, data_source):
        self.data_source = data_source

    def display_data(self):
        data = self.data_source.get_data()
        print("Display data:", data)
```

```
class DataSource(ABC):
    @abstractmethod
    def get_data(self):
        pass

class Database(DataSource):
    def get_data(self):
        return "Data from the database"

class API(DataSource):
    def get_data(self):
        return "Data from the API"
```

Что вам это даст

1

Соблюдение Open-close принципа

2

Можно легко менять зависимости

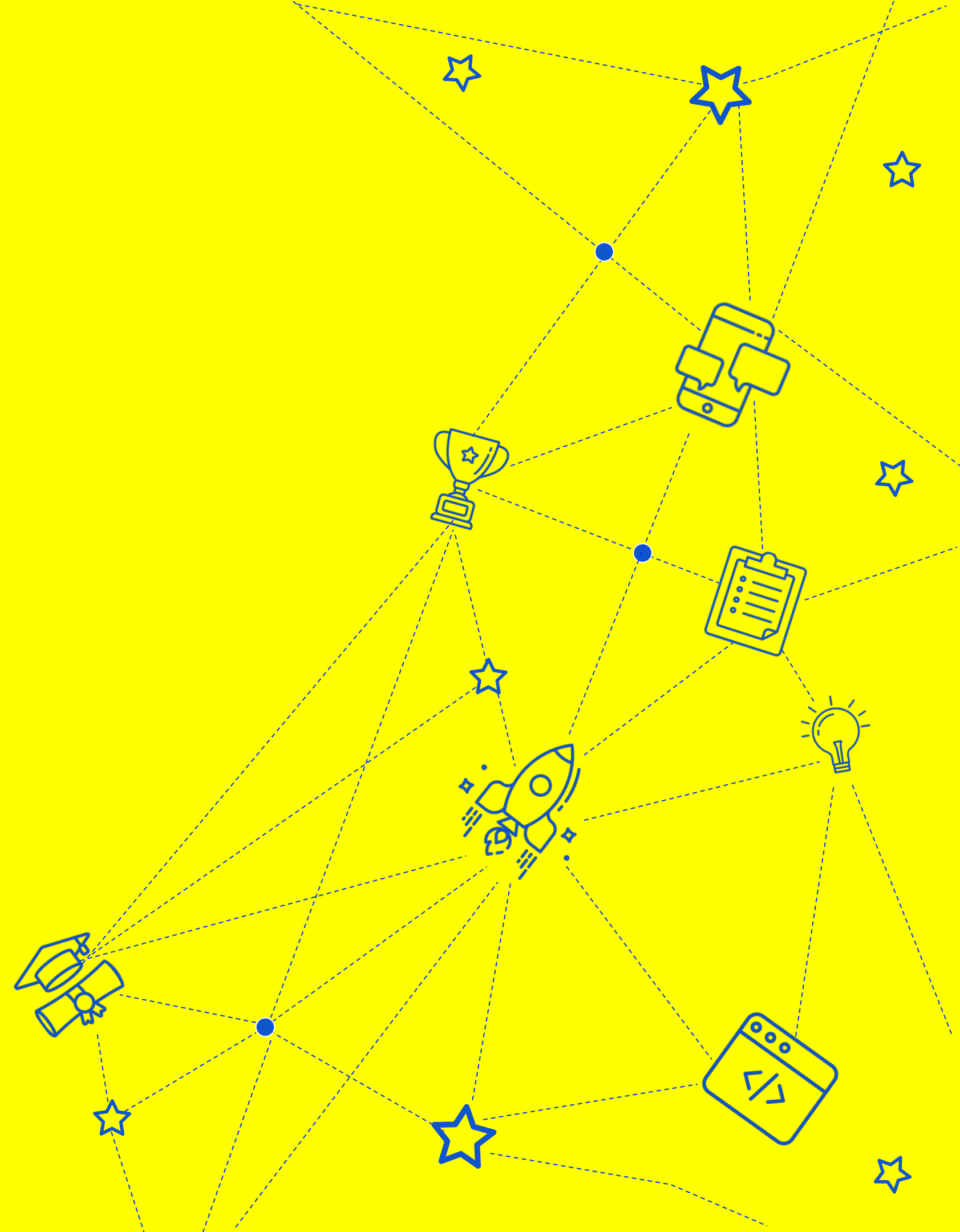
3

Высокая тестируемость кода

4

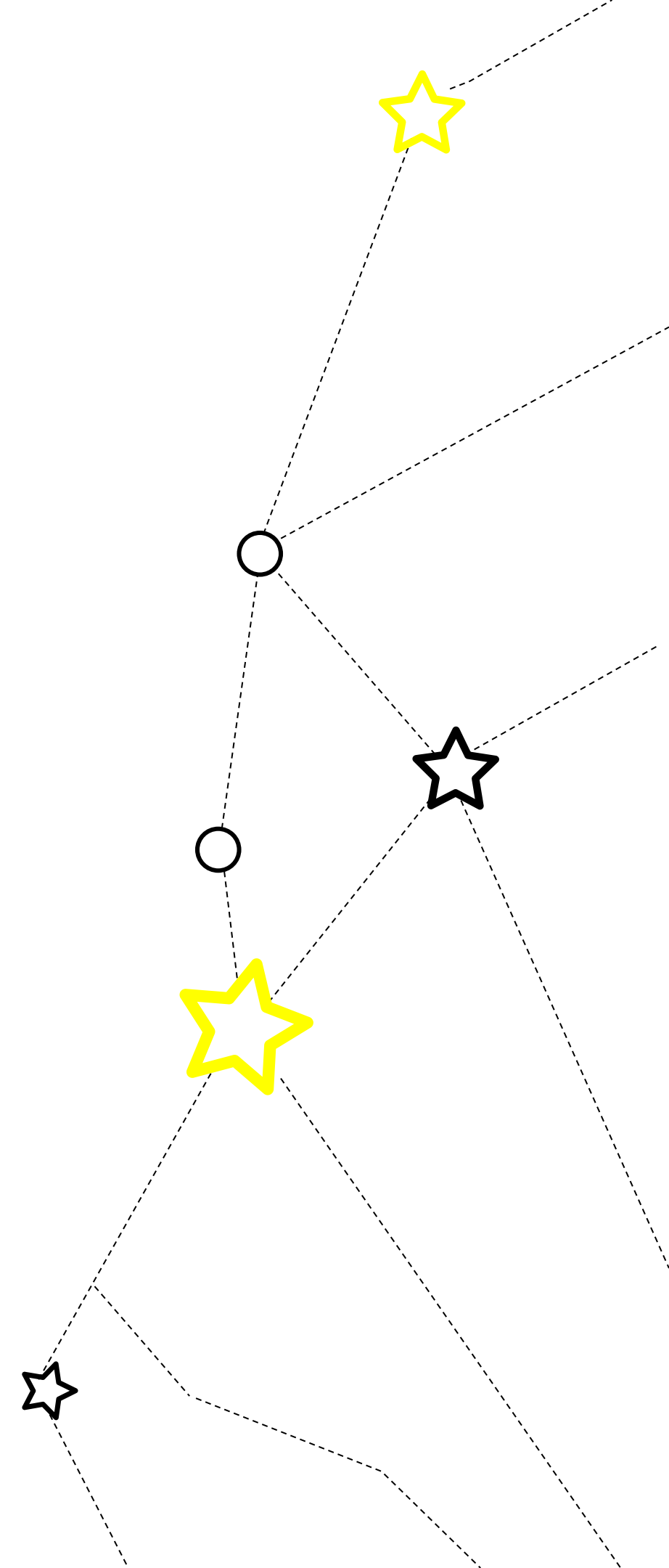
Высокая переиспользуемость кода

Воркшоп



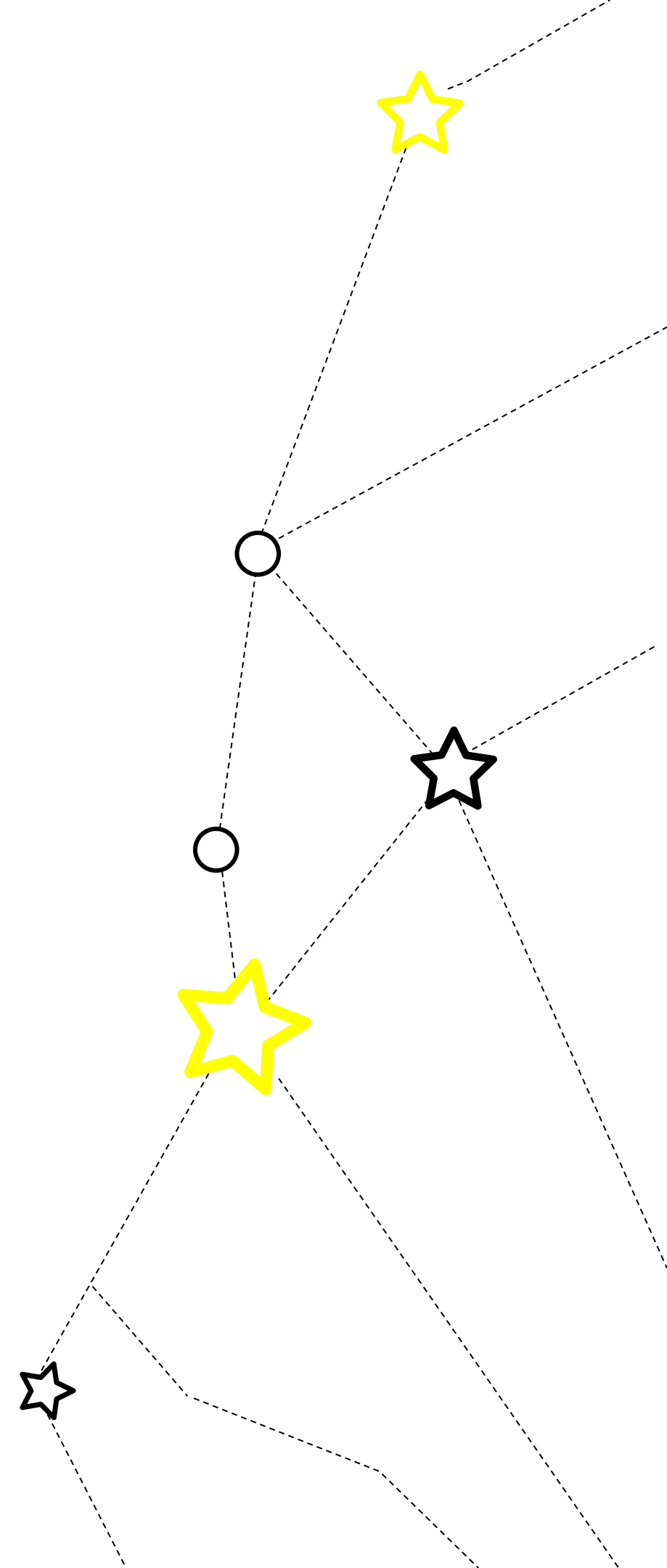
Воркшоп

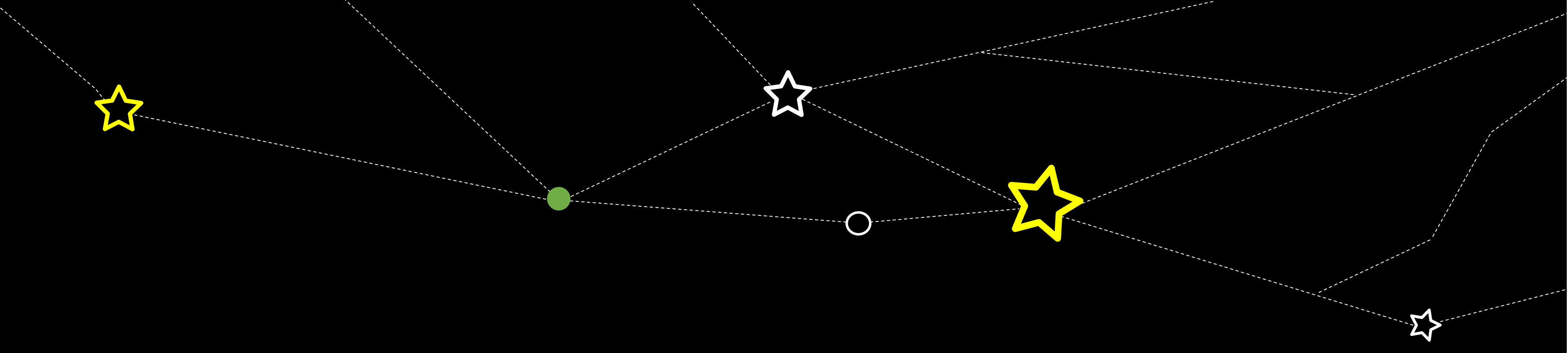
- ★ Разберем пример того, что будет если использовать и если нарушать SOLID принципы



Домашнее задание

- 1 Разобрать примеры кода
- 2 Определить, какой из принципов нарушен





СПАСИБО ЗА ВНИМАНИЕ

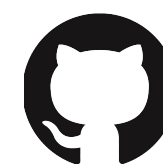
Игорь Кривонос



<https://www.linkedin.com/in/ihar-kryvanos-7066b886/>



@ikryvanos



<https://github.com/ikryvanos>